

```

# File: FindDuplicate.py
# Author: Keith Schwarz (htiek@cs.stanford.edu)
#
# An algorithm for solving the following (classic) hard interview problem:
#
# "You are given an array of integers of length  $n$ , where each element ranges
# from  $0$  to  $n - 2$ , inclusive. Prove that at least one duplicate element must
# exist, and give an  $O(n)$ -time,  $O(1)$ -space algorithm for finding some
# duplicated element. You must not modify the array elements during this
# process."
#
# This problem (reportedly) took CS legend Don Knuth twenty-four hours to solve
# and I have only met one person (Keith Ambling) who could solve it in less time
# than this.
#
# The first part of this problem - proving that at least one duplicate element
# must exist - is a straightforward application of the pigeonhole principle.
# If the values range from  $0$  to  $n - 2$ , inclusive, then there are only  $n - 1$ 
# different values. If we have an array of  $n$  elements, one must necessarily be
# duplicated.
#
# The second part of this problem - finding the duplicated element subject to
# the given constraints - is much harder. To solve this, we're going to need a
# series of nonobvious insights that transform the problem into an instance of
# something entirely different.
#
# The main trick we need to use to solve this problem is to notice that because
# we have an array of  $n$  elements ranging from  $0$  to  $n - 2$ , we can think of the
# array as defining a function  $f$  from the set  $\{0, 1, \dots, n - 1\}$  onto itself.
# This function is defined by  $f(i) = A[i]$ . Given this setup, a duplicated
# value corresponds to a pair of indices  $i \neq j$  such that  $f(i) = f(j)$ . Our
# challenge, therefore, is to find this pair  $(i, j)$ . Once we have it, we can
# easily find the duplicated value by just picking  $f(i) = A[i]$ .
#
# But how are we to find this repeated value? It turns out that this is a
# well-studied problem in computer science called cycle detection. The general
# form of the problem is as follows. We are given a function  $f$ . Define the
# sequence  $x_i$  as
#
#      $x_0 = k$  (for some  $k$ )
#      $x_1 = f(x_0)$ 
#      $x_2 = f(f(x_0))$ 
#     ...
#      $x_{n+1} = f(x_n)$ 
#
# Assuming that  $f$  maps from a domain into itself, this function will have one
# of three forms. First, if the domain is infinite, then the sequence could be
# infinitely long and nonrepeating. For example, the function  $f(n) = n + 1$  on
# the integers has this property - no number is ever duplicated. Second, the
# sequence could be a closed loop, which means that there is some  $i$  so that
#  $x_0 = x_i$ . In this case, the sequence cycles through some fixed set of
# values indefinitely. Finally, the sequence could be "rho-shaped." In this
# case, the sequence looks something like this:
#
#      $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k \rightarrow x_{k+1} \rightarrow \dots \rightarrow x_{k+j}$ 
#
#           |                                     |
#           |                                     |
#           +-----+
#
# That is, the sequence begins with a chain of elements that enters a cycle,
# then cycles around indefinitely. We'll denote the first element of the cycle
# that is reached in the sequence the "entry" of the cycle.
#
# For our particular problem of finding a duplicated element in the array,
# consider the sequence formed by starting at position  $n - 1$  and then
# repeatedly applying  $f$ . That is, we start at the last position in the array,
# then go to the indicated index, repeating this process. My claim is that
# this sequence is rho-shaped. To see this, note that it must contain a cycle
# because the array is finite and after visiting  $n$  elements, we necessarily

```

```

# must visit some element twice. This is true no matter where we start off in
# the array. Moreover, note that since the array elements range from 0 to
# n - 2 inclusive, there is no array index that contains n - 1 as a value.
# Consequently, when we leave index n - 1 after applying the function f one
# time, we can never get back there. This means that n - 1 can't be part of a
# cycle, but if we follow indices starting there we must eventually hit some
# other node twice. The concatenation of the chain starting at n - 1 with the
# cycle it hits must be rho-shaped.
#
# Moreover, think about the node we encounter that starts at the entry of the
# cycle. Since this node is at the entry of the cycle, there must be two
# inputs to the function f that both result in that index being generated. For
# this to be possible, it must be that there are indices  $i \neq j$  with
#  $f(i) = f(j)$ , meaning that  $A[i] = A[j]$ . Thus the index of the entry of the
# cycle must be one of the values that is duplicated in the array.
#
# There is a famous algorithm due to Robert Floyd that, given a rho-shaped
# sequence, finds the entry point of the cycle in linear time and using only
# constant space. This algorithm is often referred to as the "tortoise and
# hare" algorithm, for reasons that will become clearer shortly.
#
# The idea behind the algorithm is to define two quantities. First, let c be
# the length of the chain that enters the cycle, and let l be the length of the
# cycle. Next, let  $l'$  be the smallest multiple of l that's larger than c.
# I claim that for any rho-shaped sequence  $l'$  defined above, that
#
#      $x_{l'} = x_{2l'}$ 
#
# The proof is actually straightforward and very illustrative - it's one of my
# favorite proofs in computer science. The idea is that since  $l'$  is at least
# c, it must be contained in the cycle. Moreover, since  $l'$  is a multiple of
# the length of the loop, we can write it as  $ml$  for some constant m. If we
# start at position  $x_{l'}$ , which is inside the loop, then take  $l'$  more steps
# forward to get to  $x_{2l'}$ , then we will just walk around the loop m times,
# ending up right back where we started.
#
# One key trick of Floyd's algorithm is that even if we don't explicitly know l
# or c, we can still find the value  $l'$  in  $O(l')$  time. The idea is as follows.
# We begin by keeping track of two values "slow" and "fast," both starting at
#  $x_0$ . We then iteratively compute
#
#     slow = f(slow)
#     fast = f(f(fast))
#
# We repeat this process until we find that slow and fast are equal to one
# another. When this happens, we know that  $slow = x_j$  for some j, and
#  $fast = x_{2j}$  for that same j. Since  $x_j = x_{2j}$ , we know that j must be at
# least c, since it has to be contained in the cycle. Moreover, we know that j
# must be a multiple of l, since the fact that  $x_j = x_{2j}$  means that taking j
# steps while in the cycle ends up producing the same result. Finally, j must
# be the smallest multiple of l greater than c, since if there were a smaller
# multiple of l greater than c then we would have reached that multiple before
# we reached j. Consequently, we must have that  $j = l'$ , meaning that we can
# find  $l'$  without knowing anything about the length or shape of the cycle!
#
# To complete the construction, we need to show how to use our information
# about  $l'$  to find the entry to the cycle (which is at position  $x_c$ ). To do
# this, we start off one final variable, which we call "finder," at  $x_0$ . We
# then iteratively repeat the following:
#
#     finder = f(finder)
#     slow   = f(slow)
#
# until finder = slow. We claim that (1) the two will eventually hit each
# other, and (2) they will hit each other at the entry to the cycle. To see
# this, we remark that since slow is at position  $x_{l'}$ , if we take c steps
# forward, then we have that slow will be at position  $x_{l' + c}$ . Since  $l'$  is
# a multiple of the loop length, this is equivalent to taking c steps forward,
# then walking around the loop some number of times back to where you started.

```

```

# In other words,  $x_{l' + c} = x_c$ . Moreover, consider the position of the
# finder variable after  $c$  steps. It starts at  $x_0$ , so after  $c$  steps it will be
# at position  $x_c$ . This proves both (1) and (2), since we've shown that the
# two must eventually hit each other, and when they do they hit at position  $x_c$ 
# at the entry to the cycle.
#
# The beauty of this algorithm is that it uses only  $O(1)$  external memory to
# keep track of two different pointers - the slow pointer, and then the fast
# pointer (for the first half) and the finder pointer (for the second half).
# But on top of that, it runs in  $O(n)$  time. To see this, note that the time
# required for the slow pointer to hit the fast pointer is  $O(l')$ . Since  $l'$  is
# the smallest multiple of  $l$  greater than  $c$ , we have two cases to consider.
# First, if  $l > c$ , then this is  $l$ . Otherwise, if  $l < c$ , then we have that
# there must be some multiple of  $l$  between  $c$  and  $2c$ . To see this, note that
# in the range  $c$  and  $2c$  there are  $c$  different values, and since  $l < c$  at least
# one of them must be equal to  $0 \bmod l$ . Finally, the time required to find the
# start of the cycle from this point is  $O(c)$ . This gives a total runtime of at
# most  $O(c + \max\{l, 2c\})$ . All of these values are at most  $n$ , so this algorithm
# runs in time  $O(n)$ .

```

```

def findArrayDuplicate(array):
    assert len(array) > 0

    # The "tortoise and hare" step. We start at the end of the array and try
    # to find an intersection point in the cycle.
    slow = len(array) - 1
    fast = len(array) - 1

    # Keep advancing 'slow' by one step and 'fast' by two steps until they
    # meet inside the loop.
    while True:
        slow = array[slow]
        fast = array[array[fast]]

        if slow == fast:
            break

    # Start up another pointer from the end of the array and march it forward
    # until it hits the pointer inside the array.
    finder = len(array) - 1
    while True:
        slow = array[slow]
        finder = array[finder]

        # If the two hit, the intersection index is the duplicate element.
        if slow == finder:
            return slow

```